Empaquetar el modelo del proyecto como API REST en contenedor Podman

TC5061 — Operaciones de aprendizaje automático (Gpo 10) · Actividad 8

Rolando Calderón Carballo · A01840537 · Trabajo individual

Repositorio: github.com/a01840537/TC5061-Proyecto-Equipo32-rolando · **Fecha:** 7 de noviembre de 2026

1. Alcance

El modelo champion de la pregunta P1 (demanda diaria de viajes por estación) dejó de vivir en el entorno de experimentación y quedó disponible como un servicio HTTP: una API REST construida con **FastAPI**, con validación de entradas con **Pydantic**, empaquetada en una imagen **OCI** con un *Containerfile* y ejecutada con **Podman** (modo rootful, máquina liviana de 1 CPU / 2 GB). Todo se integró al repositorio por rama y pull request (`feat/api-contenedor` → PR #3 → `main`), conservando el trabajo de las entregas anteriores.

2. Recuperación del modelo desde el Model Registry

El exportador `mibici/api/exportar.py` recupera el modelo **por nombre y alias** — `models:/mibici-equipo32-p1-demanda@champion` (versión 1) — desde el Model Registry de MLflow, descarga sus artefactos a `models/api/p1/` y escribe `linaje.json` con la trazabilidad completa. El Containerfile empaqueta ese directorio dentro de la imagen, de modo que el contenedor es **autocontenido**: no depende de que el servidor de MLflow esté vivo en tiempo de ejecución. El endpoint `GET /info` expone ese linaje, respondido por el propio contenedor:


```
{"nombre": "mibici-equipo32-p1-demanda", "alias": "champion", "version": "1",
 "run_id": "aed1d7daaa8d4efba7347260fc5c19ac",
 "git_commit": "b8675e0a46c98ef93296949c6bb9ed6321881f01",
 "data_version": "7aa5e436d887", "test_mae": 5.4098, "exportado_en": "2026-11-
07T20:55:08+00:00"}
```


> **Justificación de la opción elegida:** la alternativa de conectar el contenedor al registry en tiempo de ejecución (`MLFLOW_TRACKING_URI=http://host.containers.internal:5000`) se descartó porque acopla el servicio a un servidor local encendido; empaquetar el modelo exportado hace la imagen reproducible en cualquier entorno, y la trazabilidad al registry queda garantizada por `linaje.json` y `/info` (versión, run, commit de git y versión de datos).

3. API REST con FastAPI y validación Pydantic

Método	Ruta	Descripción
GET	`/health`	Prueba de vida del servicio y del modelo cargado
GET	`/info`	Linaje del modelo servido (registry, versión, run, commit, datos)
POST	`/predict`	Predicción de viajes del día siguiente para una estación (JSON)
GET	`/docs`	Documentación interactiva (Swagger UI) generada por FastAPI

El esquema de entrada (`ObservacionDemanda`) replica exactamente las 12 variables del modelo (`P1_FEATURES`) con validaciones de tipo, rango (p. ej. `0 ≤ dia_semana ≤ 6` , `capacidad > 0` , conteos no negativos) y coherencia entre campos (`fin_semana` debe corresponder a `dia_semana`); una entrada inválida responde `422` con el detalle del error. La suite `tests/test_api.py` (8 pruebas) garantiza que el contrato de la API no se desalinee de las variables reales del modelo; con ella el repositorio llega a **34 pruebas en verde** (Figura 1).

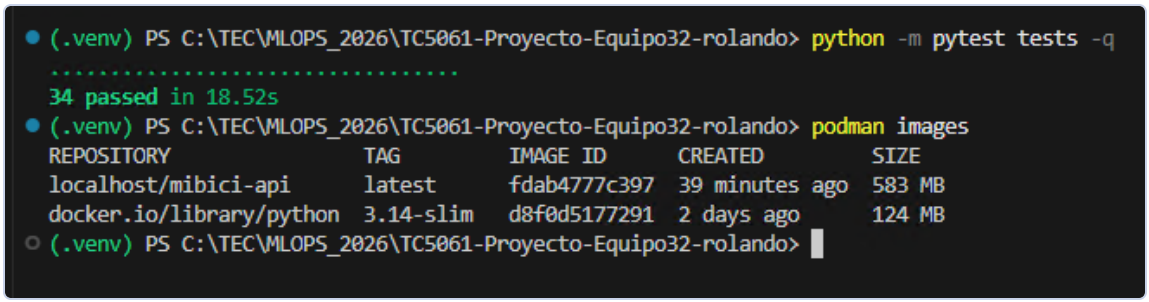


Figura 1. Suite completa tras integrar la API: 34 pruebas aprobadas; imagen `localhost/mibici-api` construida (583 MB sobre `python:3.14-slim`).

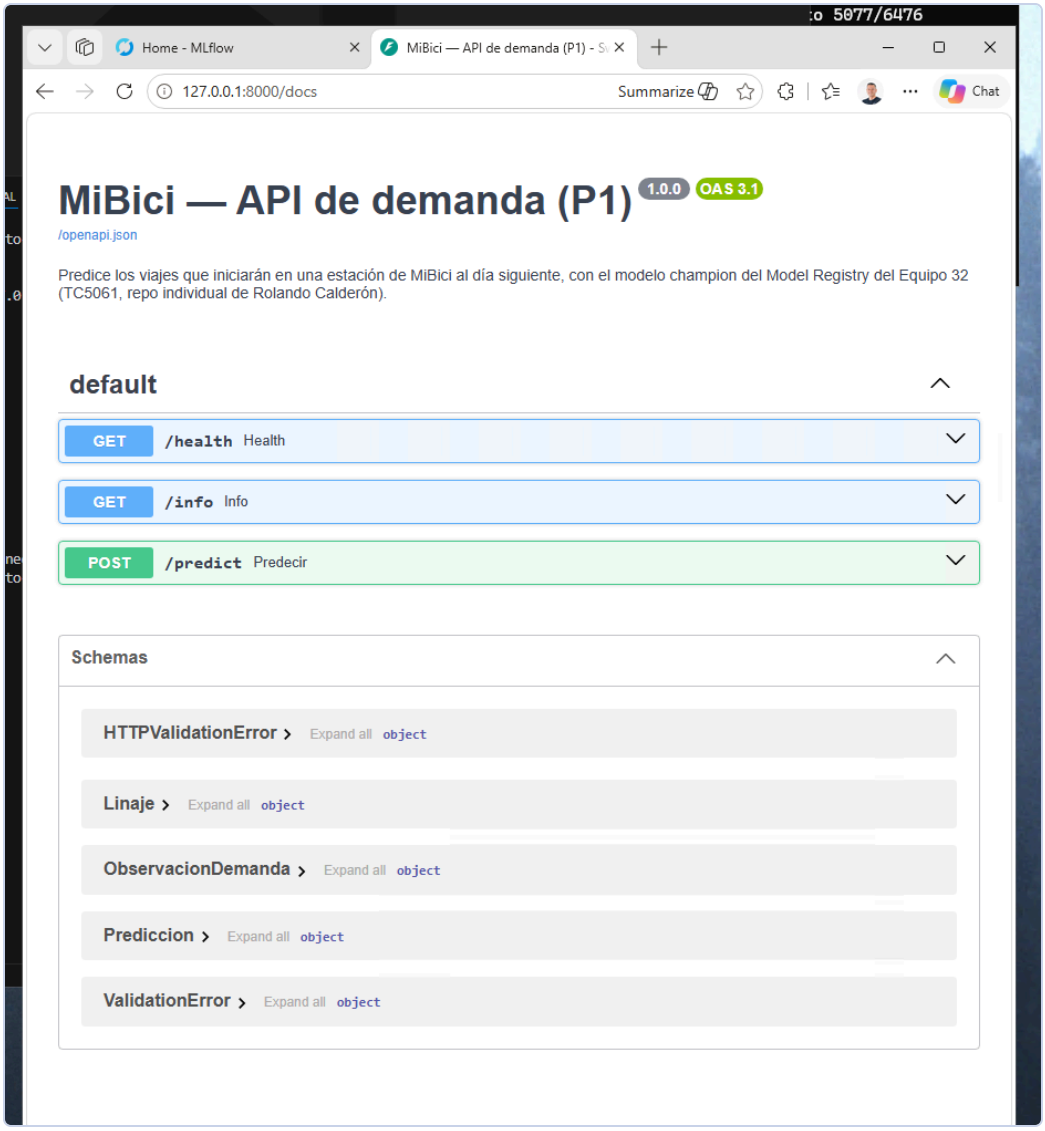


Figura 2. Documentación interactiva del servicio (`/docs`) servida por el contenedor en `127.0.0.1:8000`: endpoints y esquemas Pydantic.

4. Contenerización (Containerfile OCI + Podman)

```
FROM docker.io/library/python:3.14-slim       # misma versión de Python que el entorno de
entrenamiento
COPY requirements_api.txt → pip install       # dependencias mínimas pineadas (sin
matplotlib/folium/mlflow completo)
COPY mibici (config, features, api)           # solo el código que la API necesita
COPY models/api/p1/ → /app/modelo             # modelo champion exportado + linaje.json
USER apiuser                                  # el servicio no corre como root
CMD uvicorn mibici.api.app:app --host 0.0.0.0 --port 8000
```

- `requirements_api.txt` pinea las **mismas versiones** del entorno donde se entrenó el modelo (scikit-learn 1.9.1, pandas 3.0.6, numpy 2.5.4, skops 0.15.0, mlflow-skinny 3.17.2), para que la deserialización y las predicciones sean idénticas.
- Construcción y ejecución: `podman build -t mibici-api .` y `podman run -d --name mibici-api -p 8000:8000 mibici-api`. El mismo Containerfile funciona con Docker (estándar OCI).
- Hallazgos de Podman en Windows documentados en el README: el modo rootless no publicaba puertos en este entorno (se cambió a `--rootful`) y el proxy corporativo exige `curl --noproxy "*"` hacia `127.0.0.1`.

5. Prueba del servicio en el contenedor

Con el contenedor arriba (`podman ps`: `0.0.0.0:8000->8000/tcp`), los tres endpoints responden desde Windows (Figura 3) y el endpoint de predicción responde a la petición de prueba tanto por curl como desde Swagger (Figura 4):

```
POST /predict  {"dia_semana": 2, "fin_semana": 0, "feriado": 0, "lag_1": 48, "lag_2": 51,
                "lag_7": 45, "lag_14": 50, "media_7d": 47.3, "media_28d": 44.9,
                "media_mismo_dia_4sem": 46.5, "dias_activa": 180, "capacidad": 24}
→ 200 OK       {"viajes_predichos": 48.54, "modelo": "mibici-equipo32-p1-demanda",
"version": "1"}
```

> **Reproducibilidad verificada:** la misma petición ejecutada contra la API corriendo directamente en el entorno local (uvicorn, sin contenedor) y contra el contenedor Podman devuelve exactamente la misma predicción (48.54 viajes) — el empaquetado no altera el comportamiento del modelo, que era el objetivo de la actividad.

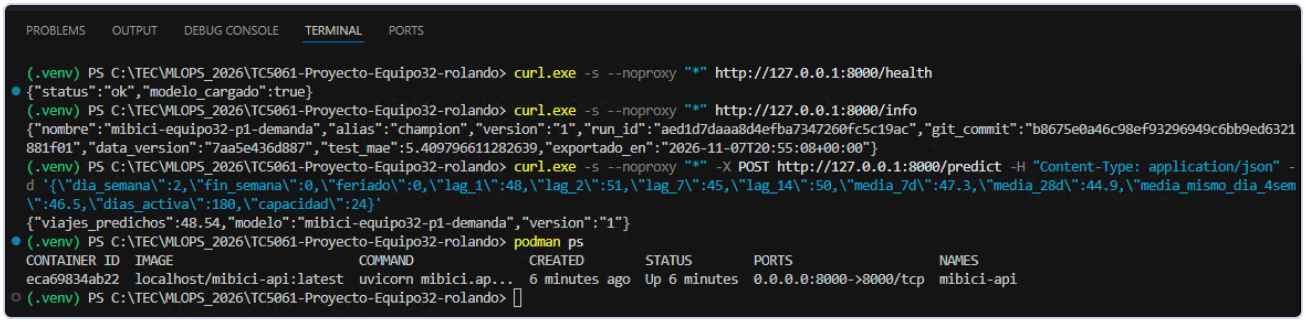


Figura 3. Peticiones de prueba al contenedor: `/health`, `/info` (linaje del registry) y `/predict` (200 OK), con `podman ps` mostrando el contenedor en ejecución y el puerto publicado.

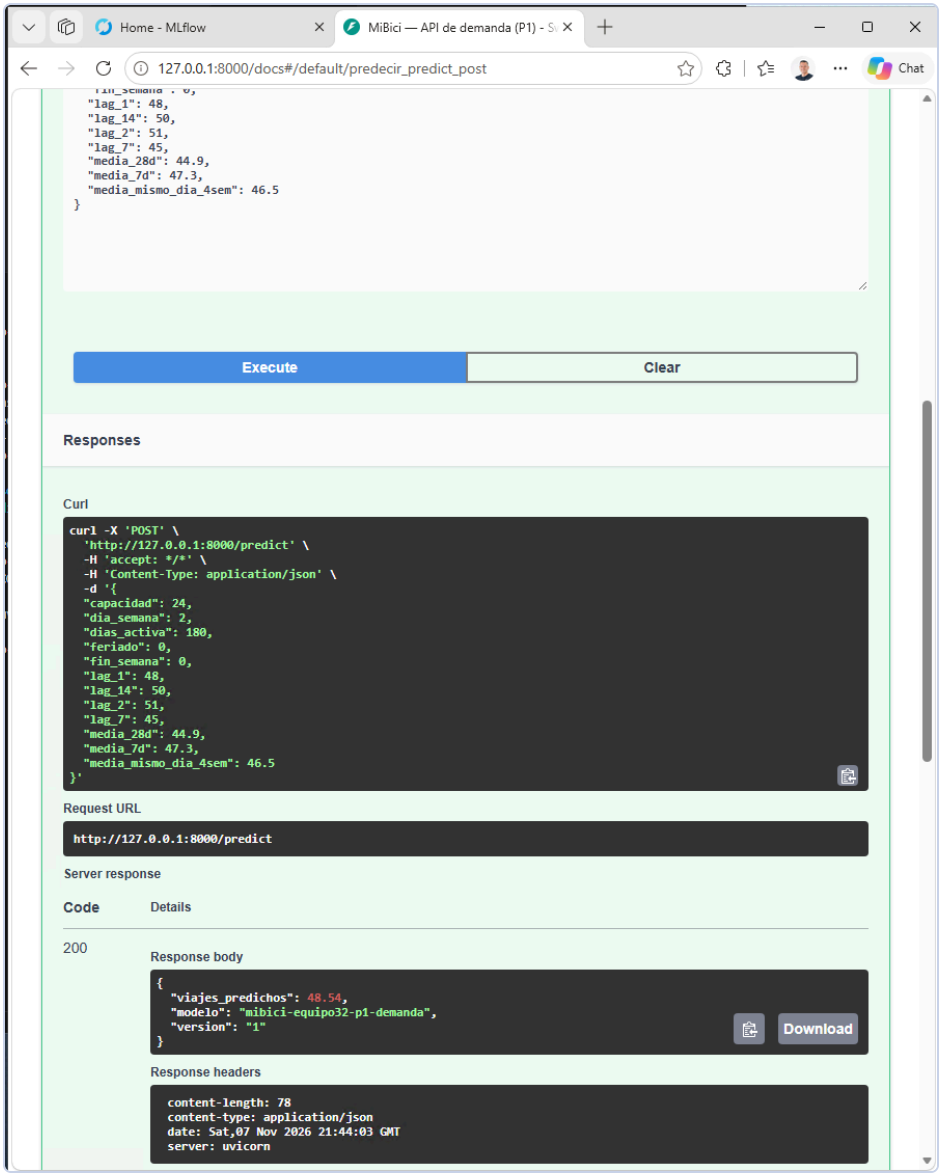


Figura 4. `POST /predict` ejecutado desde Swagger UI contra el contenedor: respuesta 200 con la predicción en JSON (48.54 viajes) servida por uvicorn.

6. Cierre

El repositorio queda con la primera versión desplegable del sistema MiBici: el champion del Model Registry servido como API REST validada, empaquetado en una imagen OCI reproducible y documentado en la sección «Actividad 8» del README (arquitectura, comandos de construcción y consumo, ejemplo de petición/respuesta y justificación de la estrategia de despliegue en tiempo real). Sobre esta base se montará la arquitectura de microservicios y la automatización de las siguientes semanas.